Foreword

Welcome, brave qubit wrangler. Before you lies a subject designed to gently, or perhaps not so gently, push you into the strange and wonderful world of quantum algorithms.

A quick pep talk. Quantum computing is weird. Your bits can be 0, 1, or *both simultaneously with complex coefficients whose squared moduli sum to one*. Gates are reversible. You cannot copy a qubit. Measurement destroys information. Phase is invisible until it interferes with itself. And somehow, by carefully orchestrating this controlled chaos, you will make an algorithm guess a secret in a single query that a classical computer would need n tries to crack. If at any point you feel like you are cheating physics, good, that means you are paying attention.

A few survival tips:

- **Q# is little-endian.** `qs[0]` is the *least* significant bit. This will bite you at least once. Probably twice. Keep a scrap of paper nearby and write down what $|101\rangle$ means for you before every exercise.
- **Ancillas must come home clean.** Any qubit you allocate with `use` must be returned to $|0\rangle$ before it goes out of scope. If you forget, the compiler will remind you. Loudly.
- **Read the hints only after trying.** They are at the end of each exercise on purpose. The satisfaction of figuring it out yourself is half the point.
- **When in doubt,** `DumpMachine()`**.** It is the quantum equivalent of `printf` debugging, except it shows you superpositions and phases. Use it shamelessly.

Good luck. May your amplitudes interfere constructively.

Testing your code: writing a `Main`

Before diving in, a quick word on how to actually run and test your code. Q# needs a single entry point to execute. You declare it with the `@EntryPoint()` attribute placed just above an operation. In VS Code with the Q# extension, a "Run" button (play icon) appears above any such operation, and clicking it shows the measurement results together with any `DumpMachine` output in the panel below. If several operations in your project are tagged `@EntryPoint()`, the compiler will complain, since only one can be the entry point at a time.

A minimal template

Here is a bare-bones test harness you can drop at the bottom of any `.qs` file:

```
import Std.Diagnostics.DumpMachine;

@EntryPoint()
operation Main() : Unit {
    // 1. Allocate qubits
    use qs = Qubit[3];

    // 2. Put them in some interesting state
    ApplyToEach(H, qs);

    // 3. Call the operation you want to test
    PhaseOracle(qs);

    // 4. Inspect the quantum state
    DumpMachine();

    // 5. Clean up before leaving
    ResetAll(qs);
}
```

The five steps above are the canonical skeleton: **allocate, prepare, run, inspect, clean up**. Forget the last step and the compiler will yell at you, since qubits must always be returned to $|0\rangle$ before being released.

Inspection tools

Two operations will be your best friends while debugging:

- `DumpMachine()` prints the full state vector: every basis state with its amplitude (magnitude and phase). Use it to *see* what your circuit actually does.
- `MResetEachZ(qs)` measures every qubit in the Z basis and resets it to $|0\rangle$ in one call. It returns a `Result[]`, which is what Deutsch–Jozsa and Bernstein–Vazirani are supposed to give back.

Keep in mind that `DumpMachine` is a *simulator-only* cheat code: it reads the underlying statevector directly, which a real quantum computer would never let you do. On a real device, you

would only ever have access to measurement outcomes. Each exercise below comes with its own sanity check in a **Test** box, so you can verify your implementation as you go.

1 Oracle

These exercises are found in the `Oracle.qs` file

1.1 Marking Oracle

Let's begin easy.

For the first oracle, we will mark the state $|101\rangle$. It means that given our oracle O_f and $|\psi\rangle$ a 3 qubits register, we have the following:

$$O_f(|\psi\rangle \otimes |0\rangle) = \begin{cases} |\psi\rangle \otimes |1\rangle & \text{if } |\psi\rangle = |101\rangle \\ |\psi\rangle \otimes |0\rangle & \text{otherwise} \end{cases}$$

which can also be written as

$$O_f(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |0 \oplus f(\psi)\rangle \quad \text{with} \quad \begin{cases} f(\psi) = 1 & \text{if } |\psi\rangle = |101\rangle \\ f(\psi) = 0 & \text{otherwise} \end{cases}$$

Complete the `MarkingOracle(qs: Qubit[], aux: Qubit): Unit` function for this exercise.

Hint. Rather than writing a long chain of controlled-X gates guarded by X flips, have a look at the Q# standard library: there is a built-in operation that applies a gate whenever a qubit register encodes a *specific integer*. You only need to figure out which integer corresponds to $|101\rangle$.
Remember that Q# uses **little-endian**: `qs[0]` is the least significant bit. So if `qs[0]=1`, `qs[1]=0`, `qs[2]=1`, what integer does the register represent?

Test. In your `Main`, prepare the register in the target state with `X(qs[0]); X(qs[2]);`, allocate an ancilla in $|0\rangle$, then call `MarkingOracle(qs, aux)` and `DumpMachine()`. You should see a single amplitude of 1 on the basis state where both `qs` encodes $|101\rangle$ *and* the ancilla is $|1\rangle$. Try the same with another initial state (say $|100\rangle$): this time the ancilla should remain at $|0\rangle$.

1.2 Phase Kickback

The *phase kickback* is a fundamental mechanism that turns a marking oracle (like the one from the previous section) into a phase oracle. It is the key ingredient that makes algorithms such as Deutsch–Jozsa, Bernstein–Vazirani and Grover possible.

The problem. A marking oracle writes the result $f(x)$ into an ancillary qubit through a XOR:

$$O_f \, |x\rangle \, |y\rangle = |x\rangle \, |y \oplus f(x)\rangle$$

The information about $f(x)$ therefore ends up in the ancilla, whereas in order to exploit quantum interference we would like it to directly affect the input register $|x\rangle$.

The trick. We initialize the ancilla in the state $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ (by applying X then H to $|0\rangle$) instead of $|0\rangle$. Let us compute the action of O_f on $|x\rangle\,|-\rangle$:

$$O_f\,|x\rangle\,|-\rangle = |x\rangle \cdot \frac{|0 \oplus f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}}$$

Two cases arise:

- If $f(x) = 0$: we get $|x\rangle \cdot \dfrac{|0\rangle - |1\rangle}{\sqrt{2}} = |x\rangle\,|-\rangle$
- If $f(x) = 1$: we get $|x\rangle \cdot \dfrac{|1\rangle - |0\rangle}{\sqrt{2}} = -\,|x\rangle\,|-\rangle$

Both cases can be gathered into a single formula:

$$\boxed{O_f\,|x\rangle\,|-\rangle = (-1)^{f(x)}\,|x\rangle\,|-\rangle}$$

The result. The ancilla remains unchanged in $|-\rangle$, but a phase $(-1)^{f(x)}$ appears in front of the input register $|x\rangle$. The phase has been "kicked back" from the ancilla onto the control register, hence the name. Applied to a superposition, this directly yields a phase oracle:

$$O_f\left(\sum_x \alpha_x\,|x\rangle\right)|-\rangle = \left(\sum_x (-1)^{f(x)}\alpha_x\,|x\rangle\right)|-\rangle$$

This is why the following exercises ask you to implement a phase oracle directly: it encodes the same f, but in a form that can be exploited through interference without having to manage an ancilla.

1.3 Phase Oracle

Now let's consider the phase oracle.

Instead of marking the desired state by writing the result in an ancillary qubit, the oracle will apply a phase of -1 to the marked state $|101\rangle$.

The oracle U_f acts on 3-qubit as follows:

$$U_f\,|\psi\rangle = \begin{cases} -\,|\psi\rangle & \text{if } |\psi\rangle = |101\rangle \\ |\psi\rangle & \text{otherwise} \end{cases}$$

which is equivalent to

$$U_f\,|\psi\rangle = (-1)^{f(\psi)}\,|\psi\rangle \quad \text{with} \quad \begin{cases} f(\psi) = 1 & \text{if } |\psi\rangle = |101\rangle \\ f(\psi) = 0 & \text{otherwise} \end{cases}$$

Complete the `PhaseOracle(qs: Qubit[]): Unit` function for this exercise.

Hint. You just learned the phase kickback trick: if you feed the ancilla of a marking oracle in state $|-\rangle$, the oracle automatically becomes a phase oracle. So you do *not* need to rewrite anything, you can simply reuse your `MarkingOracle` here.

Two practical points in Q#:

- You can allocate a temporary qubit with `use aux = Qubit();`. It must be returned to $|0\rangle$ before it goes out of scope.
- The `within { ... } apply { ... }` construct automatically applies the adjoint of the `within` block after the `apply` block. This is *exactly* what you want: prepare $|-\rangle$, run the oracle, and let Q# undo the preparation for you, with no `MResetZ` needed.

Test. Put the register in a uniform superposition with `ApplyToEach(H, qs)`, call `PhaseOracle(qs)`, then `DumpMachine()`. You should see all eight basis states with amplitude $\pm 1/\sqrt{8}$, with the minus sign appearing only on index 5 (which is $|101\rangle$ in little-endian). If the ancilla you allocated internally is still entangled with `qs` in the dump, your `within ... apply` block did not decompute it properly.

2 Deutsch–Jozsa

In the Deutsch–Jozsa algorithm, we use a Boolean function.

$$f : \{0,1\}^n \to \{0,1\}$$

with the promise that f is either:

- **Constant**: $f(x) = 0$ for all x, or $f(x) = 1$ for all x.
- **Balanced**: $f(x) = 0$ for exactly half of the inputs, and $f(x) = 1$ for the other half.

The goal is to determine whether f is constant or balanced using a single query to a quantum oracle. The quantum oracle O_f acts as follows:

$$O_f(|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \oplus f(x)\rangle$$

- x: n-qubit input register
- y: 1-qubit output register (usually initialized to $|1\rangle$)
- $f(x)$: the Boolean function we are testing

However, in this algorithm and many others, the marking oracle O_f is used with phase kickback as a phase oracle U_f. Be careful as Wikipedia uses a half phase half marking oracle method, which might not work on Q# as it doesn't leave the ancillary qubit as $|0\rangle$ without using `MResetZ()`. You will be encouraged to use the phase oracle method in those exercises.

2.1 Constant oracle

A constant oracle implements a function $f : \{0,1\}^n \to \{0,1\}$ such that $f(x) = c$ for all $x \in \{0,1\}^n$, where $c \in \{0,1\}$ is a fixed value.

In this case, we consider the constant function: $f(x) = 1 \ \ \forall x \in \{0,1\}^n$

Here, you will implement this oracle as a phase oracle such that $U_f |x\rangle = (-1)^{f(x)} |x\rangle$ in the function `operation ConstOracle(qs: Qubit[]) : Unit`

Hint. The function $f(x) = 1$ does not depend on x at all, so ask yourself whether you really need to touch the `qs` register. Look at the phase kickback formula: for $f(x) = 1$ everywhere, the oracle should produce a global phase of -1 on every input.

Recall that $|-\rangle$ is an eigenstate of X with eigenvalue -1, that is, $X |-\rangle = - |-\rangle$. Can you use that on an ancilla prepared in $|-\rangle$?

Test. Prepare a uniform superposition with `ApplyToEach(H, qs)`, call `ConstOracle(qs)`, then `DumpMachine()`. You should see *every* amplitude flip to $-1/\sqrt{2^n}$. That global minus sign is invisible on its own (a global phase cannot be measured), but combined with the Hadamard layer in Deutsch–Jozsa it will produce $- |0\rangle^{\otimes n}$, which measures as $|0\rangle^{\otimes n}$ with probability 1.

2.2 Balanced oracle

In the context of Deutsch–Jozsa, a balanced function is such that: $f(x) = 0$ for exactly half of the inputs, and $f(x) = 1$ for the other half.

In this case, we consider the balanced function: $f(x) = x \bmod 2 \ \ \forall x \in \{0,1\}^n$

Here, you will implement this oracle as a phase oracle such that $U_f |x\rangle = (-1)^{f(x)} |x\rangle$ in the function `operation BalancedOracle(qs: Qubit[]) : Unit`

Hint. Write $x \bmod 2$ in terms of the bits of x: it is simply the least significant bit. In Q#'s little-endian convention, which qubit of `qs` is the LSB?

Once you know which qubit controls f, you only need to kick back a phase when that qubit is $|1\rangle$. What is the simplest two-qubit gate that applies an operation conditionally on one qubit?

Test. With the register in a uniform superposition, calling `BalancedOracle(qs)` should flip the sign of exactly half the amplitudes: those whose index is odd in little-endian. For $n = 3$, the `DumpMachine` output should show $+1/\sqrt{8}$ on indices $0, 2, 4, 6$ and $-1/\sqrt{8}$ on indices $1, 3, 5, 7$.

2.3 Deutsch–Jozsa circuit

Algorithm steps (for n-qubit input) with Phase Oracle:

1. Initialize n qubits in state $|0\rangle^{\otimes n}$
$$|0\rangle^{\otimes n}$$
2. Apply Hadamard gates to all qubits: Apply Hadamard gates to all qubits to create a uniform superposition:
$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$
3. Apply the Oracle U_f: We now apply the phase oracle U_f, which encodes the function f into the phase of each basis state:
$$U_f \left(\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \right) = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle$$

4. Apply the Hadamard gates:

$$H^{\otimes n}\left(\frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}(-1)^{f(x)}\,|x\rangle\right)=\sum_{z\in\{0,1\}^n}\left[\frac{1}{2^n}\sum_{x=0}^{2^n-1}(-1)^{f(x)\oplus x\cdot z}\right]|z\rangle$$

5. Measure the n qubits

Interpretation:

- If f is constant, the result is always $|0\rangle^{\otimes n}$.
- If f is balanced, the result is never $|0\rangle^{\otimes n}$.

Here, you will implement the Deutsch–Jozsa circuit with the constant oracle and the balanced oracle. You will use the constant oracle in `operation ConstantDeutschJosza(): Result[]` and the balanced one in `operation BalancedDeutschJosza(): Result[]`

Hint. The circuit has the structure $H^{\otimes n} \to U_f \to H^{\otimes n}$. Since H is its own adjoint ($H^\dagger = H$), applying $H^{\otimes n}$ before and after the oracle is *exactly* what the Q# pattern `within { ... } apply { ... }` does for you automatically.
Two gotchas:

- The `within` block requires an adjointable operation, so use `ApplyToEachA`, not `ApplyToEach`.
- The subject asks for *two* distinct entry points, `ConstantDeutschJosza()` and `BalancedDeutschJosza()`. Do not squeeze both into a single operation with a commented line, duplicate the circuit instead.

Finally, `MResetEachZ` is the idiomatic way to measure and release the qubits in one call.

Test. Run each circuit several times. `ConstantDeutschJosza()` must always return `[Zero, Zero, Zero]`, while `BalancedDeutschJosza()` must *never* return that. If the constant version ever gives a non-zero bit, the oracle is probably not applying a true global phase. If the balanced version gives all zeros, you likely forgot one of the two Hadamard layers (or used `ApplyToEach` instead of `ApplyToEachA`, which silently skips the second layer in a `within ... apply` block).

3 Bernstein–Vazirani

This section is on purpose harder to understand by yourself. Make sure you've covered every exercise beforehand.

The problem

Imagine a black box that hides a secret bit string $s \in \{0,1\}^n$. The only way to interact with the box is to feed it an input x, and it spits out one bit: the bitwise inner product of s and x modulo 2, that is,

$$f(x) = s \cdot x = s_0x_0 \oplus s_1x_1 \oplus \cdots \oplus s_{n-1}x_{n-1}$$

How fast can you find s?

Classically, each query gives you exactly one bit of information, and you need to cleverly choose your inputs to isolate each s_i. The natural strategy is to query $x = 100\ldots0$, which returns s_0, then $x = 010\ldots0$ to get s_1, and so on. In total: n queries, one per bit of the secret. You cannot do better, since each query returns a single bit and the secret has n bits of information.

Quantumly, Bernstein–Vazirani extracts the *entire* secret with **a single query** to the oracle. No matter how large n is. That is already a noticeable speedup over the classical approach, and the circuit that achieves it is remarkably simple: $H^{\otimes n}$, one oracle call, $H^{\otimes n}$, measure. That is literally it.

The trick is the same interference pattern we saw in Deutsch–Jozsa, but tuned more precisely. Phase kickback puts the oracle's answer into the phases of a uniform superposition, and the final Hadamard layer turns those phases into a single basis state, namely $|s\rangle$.

3.1 Secret oracle

The oracle implements the following function, given a secret bit string s made of n bits:

$$f : \begin{cases} \{0,1\}^n \longrightarrow \{0,1\} \\ x \longmapsto \bigoplus_{i=0}^{n-1} s_i x_i \end{cases}$$

Here, you will implement this oracle given an integer as an input which corresponds to your secret. Keep in mind that Q# is, like this subject, in little-endian. You will find more details above the function to do as a comment.

The function to implement is the following: `operation Oracle(secret: Int, qs: Qubit[]): Unit`.

Hint. You want to kick back a phase $(-1)^{s_i x_i}$ for each bit position i, and let them multiply together. Since $(-1)^{a \oplus b} = (-1)^a \cdot (-1)^b$, you do not need one big controlled operation, just n small ones that each contribute their own phase.

Concretely: loop over the bits of the secret. Whenever bit i of s is 1, add a `CX(qs[i], aux)` to the circuit. Whenever it is 0, skip qubit i entirely. With the ancilla in $|-\rangle$, each such `CX` contributes a factor $(-1)^{x_i}$ exactly when $s_i = 1$, and the total phase ends up being $(-1)^{s \cdot x}$.

To extract the bits of an integer `secret` in Q#, a classical `mutable` variable together with `s % 2` and `s / 2` works fine. Watch out for off-by-one errors in the loop bounds, `IndexRange(qs)` is safer than writing the range by hand.

Test. Try `secret = 5` with 3 qubits. After `ApplyToEach(H, qs)` and `Oracle(5, qs)`, `DumpMachine` should show amplitudes $+1/\sqrt{8}$ everywhere except on indices $1, 2, 4, 7$, which get flipped to $-1/\sqrt{8}$. These are exactly the $x \in \{0, \ldots, 7\}$ for which $s \cdot x = 101 \cdot x$ is odd. If you see a different pattern, print the bits your loop extracts from the secret and compare them to the binary representation of $5 = 101_2$ in little-endian.

3.2 Bernstein–Vazirani circuit

1. Input: $|0\rangle^{\otimes n}$ and a secret s for $f(x)$
2. $H^{\otimes n} |0\rangle^{\otimes n} = \dfrac{1}{\sqrt{2^n}} \displaystyle\sum_{i=0}^{2^n-1} |i\rangle$

3. $U_f \frac{1}{\sqrt{2^n}} \sum_{i=0}^{2^n-1} |i\rangle = \frac{1}{\sqrt{2^n}} \sum_{i=0}^{2^n-1} (-1)^{f(i)} |i\rangle$

4. $H^{\otimes n} \frac{1}{\sqrt{2^n}} \sum_{i=0}^{2^n-1} (-1)^{f(i)} |i\rangle = |s\rangle$

5. Output: Measure gives s

Why does step 4 work? The step $H^{\otimes n} \left(\sum_i (-1)^{f(i)} |i\rangle\right) = |s\rangle$ deserves a proper justification, since the whole algorithm relies on it.

Recall that a single Hadamard gate acts on a basis state as

$$H |b\rangle = \frac{1}{\sqrt{2}} \sum_{c \in \{0,1\}} (-1)^{b \cdot c} |c\rangle$$

Tensoring this over n qubits, one obtains the key identity

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{x \cdot z} |z\rangle \qquad \text{where } x \cdot z = \bigoplus_{i=0}^{n-1} x_i z_i$$

is the bitwise inner product modulo 2.

Now observe that for Bernstein–Vazirani, $f(x) = s \cdot x$ by definition. Applying $H^{\otimes n}$ to the state after the oracle gives:

$$H^{\otimes n} \left(\frac{1}{\sqrt{2^n}} \sum_x (-1)^{s \cdot x} |x\rangle \right) = \frac{1}{2^n} \sum_z \left(\sum_x (-1)^{s \cdot x} (-1)^{x \cdot z} \right) |z\rangle = \frac{1}{2^n} \sum_z \left(\sum_x (-1)^{x \cdot (s \oplus z)} \right) |z\rangle$$

The inner sum is a classic orthogonality relation: summing $(-1)^{x \cdot y}$ over all $x \in \{0,1\}^n$ gives 2^n if $y = 0$, and 0 otherwise. Applied with $y = s \oplus z$:

$$\sum_{x \in \{0,1\}^n} (-1)^{x \cdot (s \oplus z)} = \begin{cases} 2^n & \text{if } z = s \\ 0 & \text{otherwise} \end{cases}$$

So every term in the sum over z vanishes *except* the one with $z = s$, which survives with amplitude 1:

$$H^{\otimes n} \left(\frac{1}{\sqrt{2^n}} \sum_x (-1)^{s \cdot x} |x\rangle \right) = |s\rangle$$

This is why a single measurement reveals the secret s with probability 1, whereas a classical algorithm would need n queries (one per bit of s).

Here, you will implement this circuit using the previously made oracle. You are free to choose the secret for the oracle, which has to be **given as an argument and not hardcoded**, as we will test it with other values. The function to implement is the following: `operation BernsteinVazirani(): Result[]`

Hint. The structure is identical to Deutsch–Jozsa: $H^{\otimes n}$, then oracle, then $H^{\otimes n}$, then measure. Reuse the `within { ApplyToEachA(H, qs); } apply { Oracle(secret, qs); }` pattern.

Test. Pick `secret = 13` with 4 qubits. Since $13 = 1101_2$, in little-endian you should read the measurement as `[One, Zero, One, One]` (that is, `qs[0]=1, qs[1]=0, qs[2]=1, qs[3]=1`). The result should be *deterministic*: if you see different outcomes across runs, either your oracle does not implement $s \cdot x$ correctly, or you forgot the second $H^{\otimes n}$. Try a few more secrets (7, 42, and so on) to convince yourself it works for arbitrary inputs.